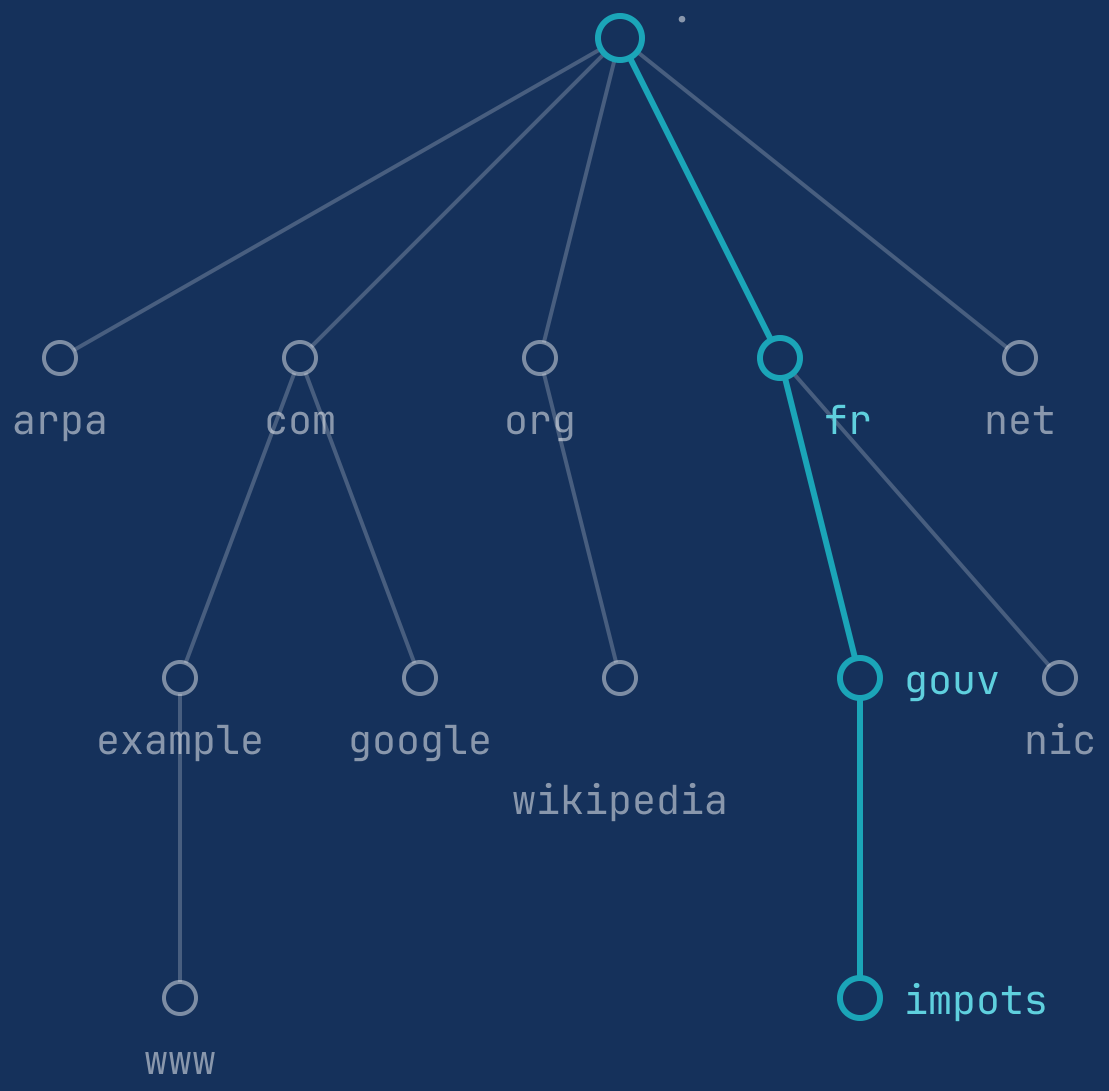
DNS Beyond the Basics

Julien MANTEAU

2026



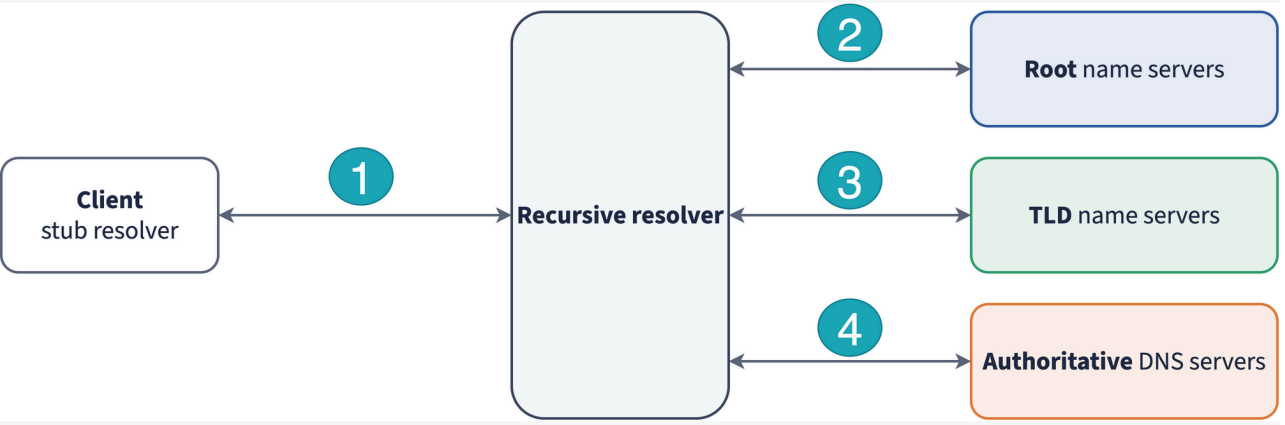
How to use this deck

- Read it **on your own** before the **Advanced** missions of Dig Quest. They unlock once 80% of the main missions are solved.
- It starts where *DNS Fundamentals* stops: you already know records, resolution, TTL, flags, NODATA and the idea of DNSSEC.
- Every slide ends with a **Try it** command. Run it: the answers change over time, and reading live output is the skill being trained.
- Each Advanced mission names the part of this deck it needs.

TRY IT `dig -v` (you need dig 9.16 or later for every command in this deck)

Agenda

1	Who answers you? Anycast, CHAOS, NSID, your resolver
2	Reading answers closely: SOA, error codes, wildcards, caches
3	Size and transport: 512 bytes, EDNS, TCP, long TXT
4	Special records and names: null MX, mail auth, SRV, NAPTR, IPv6, IDN
5	DNSSEC in practice: the chain of trust, the flags, SERVFAIL
6	Opening a zone: zone transfers and NSEC walking
7	DNS as a puzzle: encodings, chained clues, files in pieces
LAB	Dig Quest Advanced missions



PART 1

Who answers you?

Anycast, CHAOS, NSID and the resolver behind your resolver

One address, many servers: anycast

`f.root-servers.net` has **one** IP address, announced by **hundreds of sites**. Internet routing (BGP) sends your packet to the closest one. This is **anycast**, and most root servers, TLD servers and public resolvers use it.

FAST The closest copy answers

ROBUST A DDoS is spread across all the sites

BUT You can't tell which machine answered

PREDICT **From Toulouse, you ask F-root to name itself. Which city answers?**

```
❯ dig @f.root-servers.net CH TXT hostname.bind +short
"CDG.cf.f.root-servers.org"          # CDG: the Paris airport code
```

TRY IT Same query to `k.root-servers.net`, then to `l.root-servers.net`. Same city?

The CHAOS class: servers describing themselves

Every record has a **class**. You only ever saw `IN` (Internet). Two others survive:

- `CH`: **Chaosnet**, an MIT network from the 1970s
- `HS`: **Hesiod**, an MIT directory service

Nobody uses Chaosnet anymore, so servers reuse the `CH` class for **questions about themselves**:

- `version.bind`, `hostname.bind`, `authors.bind` (BIND)
- `id.server`, `version.server` (the standard names, RFC 4892)

```
❯ dig @1.1.1.1 CH TXT id.server +short
"cdg14"
❯ dig @9.9.9.9 CH TXT id.server +short
"res711.cdg"
❯ dig @k.root-servers.net CH TXT hostname.bind +short
"ns1.fr-lio.k.ripe.net"
❯ dig @a.gtld-servers.net CH TXT version.bind +short
"ATLAS"
```

Verisign answers “ATLAS” (its own software) and no version number. Knowing the exact version helps an attacker pick an exploit, so many operators hide it or return a joke.

TRY IT `dig @8.8.8.8 CH TXT id.server`: what does Google answer, and with which status?

NSID: the identity inside the answer

A `CH TXT` query is a **separate** packet: with anycast it may reach another instance than your real query did. **NSID** (RFC 5001) asks the server to add its identity to **the same answer**, as an EDNS option.

```
❯ dig @l.root-servers.net . SOA +norecurse +nsid
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 13, ADDITIONAL: 27
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; NSID: 62 67 2d 67 6f 7a 2d 61 61 ("bg-goz-aa")
```

PREDICT **L-root also names itself in a normal record (RFC 7108). Whose instance is it?**

```
❯ dig IDENTITY.L.ROOT-SERVERS.ORG TXT +short
"de-ham-aa."     # not yours: the one your resolver reached (Hamburg)
```

TRY IT Add `+nsid` to a query to `1.1.1.1` or `9.9.9.9`. Which instance answered?

The `dig ch` trap

dig reads each word on its own: a **class** name (`in`, `ch`, `hs`) becomes the class, a **type** name becomes the type. With no name left, dig asks for the root (`.`).

```
❯ dig ch NS +short
(nothing)
❯ dig in NS +short
a.root-servers.net.
b.root-servers.net.
…
m.root-servers.net.
```

- `dig ch NS` sends `. CH NS`: root servers in the Chaosnet class. Switzerland never shows up.
- `dig in NS` sends `. IN NS`: the root servers, not India's.
- **Fix:** write the name in full, with its final dot: `dig ch. NS`, `dig in. NS`.

TRY IT `dig ch. NS +short`: which organisation runs the `.ch` servers? (Hint: `nic.ch`.)

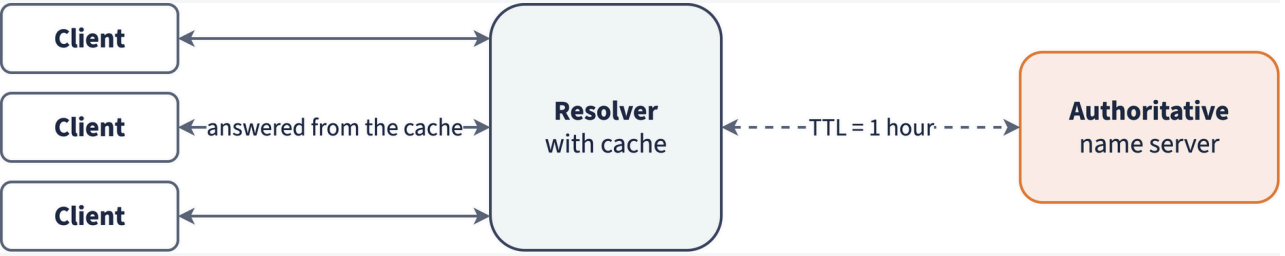
Who is my resolver, really?

Your machine asks a resolver, which may forward to another, and the authoritative server only sees the **last hop**. **“whoami”** names are served by authoritative servers that put **the source of the question** in the answer.

Ask	You learn
`dig TXT o-o.myaddr.l.google.com @8.8.8.8 +short`	IP your resolver uses to reach Google’s servers, plus the ECS it sent
`dig TXT whoami.ds.akahelp.net +short`	Same idea at Akamai: `“ns”` resolver IP, `“ecs”` subnet
`dig myip.opendns.com @resolver4.opendns.com +short`	Your own public IP (you asked OpenDNS directly)

```
❯ dig TXT o-o.myaddr.l.google.com @8.8.8.8 +short
"edns0-client-subnet 86.201.70.0/24"     # your network, sent on your behalf
"172.253.92.148"                          # a Google resolver, not you
```

TRY IT Same question through `1.1.1.1`, then `9.9.9.11`. Which ones pass your subnet (ECS) along?



PART 2

Reading answers closely

SOA fields, extended errors, wildcards and caches

The SOA, field by field

```
❯ dig iut-blagnac.fr SOA +short
brassens.iut-blagnac.fr. pinf.iutb.univ-tlse2.fr. 2026082311 3600 600 1209600 3600
```

Field	Value	Meaning
MNAME	`brassens.iut-blagnac.fr.`	Primary server of the zone
RNAME	`pinf.iutb.univ-tlse2.fr.`	Admin mailbox: the first `.` becomes `@` → pinf@iutb.univ-tlse2.fr
SERIAL	`2026082311`	Version of the zone. Common habit `YYYYMMDDnn`: 23 Aug 2026, change 11. Only a habit: `.fr` uses a counter
REFRESH / RETRY	`3600` / `600`	How often secondaries check the serial, and retry after a failure
EXPIRE	`1209600`	Secondaries stop answering after 14 days without the primary
MINIMUM	`3600`	Negative caching limit: how long “does not exist” is kept

TRY IT `dig fr SOA +short` and `dig afnic.fr SOA +short`: which serial is a date? Who to e-mail?

Why did it fail? Extended DNS Errors

`SERVFAIL` only says “something went wrong”. **Extended DNS Errors** (EDE, RFC 8914) add a code and a text in the OPT section.

Code	Meaning
6, 7, 9	DNSSEC bogus, signature expired, DNSKEY missing
15, 17	Blocked, Filtered (a DNS firewall)
18	Prohibited (you may not use this resolver)
22	No reachable authority
0	Other: read the text

```
❯ dig dnssec-failed.org @8.8.8.8
;; ⟶»HEADER«⟵ opcode: QUERY, status: SERVFAIL, id: 42839
; EDE: 9 (DNSKEY Missing): (No DNSKEY matches DS RRs of dnssec-failed.org)
```

TRY IT The same name through `1.1.1.1` and `9.9.9.9`: same status? same EDE code?

Wildcards

A record named `*.github.io` answers for **any name that does not exist** under `github.io`. The rules (RFC 4592):

- `*` only as the **leftmost** label
- an existing name **blocks** it: no record of that type gives NODATA, not the wildcard
- it does not reach below an existing name

```
❯ dig zz-not-here-42.github.io +short
185.199.108.153
185.199.109.153
…
❯ dig zz-not-here-42.wikipedia.org +short
(nothing: NXDOMAIN)
```

PREDICT **The answer shows the name you asked, never the `*`. Can you prove a wildcard was used?**

Only with DNSSEC: the RRSIG has a **labels** field. A signature with fewer labels than the name was made for `*.github.io`.

TRY IT `dig anything-you-want.blogspot.com +short`, then `dig "*.blogspot.com" +short`

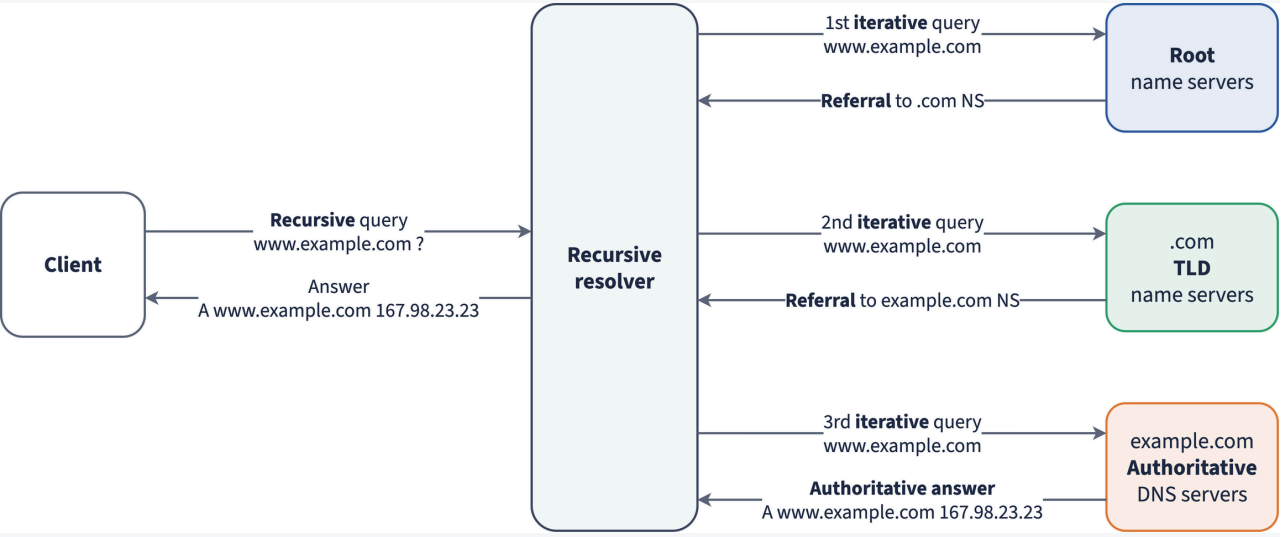
Peeking into a cache: `+norecurse`

Without the `rd` flag, you ask a resolver: **“answer only from your cache”**. If it plays along, you learn whether someone already asked for that name, and when (the TTL left).

```
❯ dig @8.8.8.8 www.google.com +norecurse +noall +answer
www.google.com.   154   IN   A   142.251.152.119      # cached, 154 s left
❯ dig @1.1.1.1 www.iut-blagnac.fr +norecurse
;; status: SERVFAIL
; EDE: 0 (Other): (no local cache to fulfill non recursion (RD=0) request)
❯ dig @9.9.9.9 www.google.com +norecurse +noall +answer
(nothing)
```

- **Cache snooping:** on a company resolver, this reveals which sites its users visit. Many resolvers refuse `rd=0` from outside.
- **Anycast again:** each instance has its own cache. Run it twice and the TTL may jump.

TRY IT Run the 8.8.8.8 query three times, a few seconds apart. Does the TTL count down smoothly?



PART 3

Size and transport

512 bytes, EDNS, the tc flag and long TXT records

512 bytes, EDNS and the `tc` flag

1. **1987** (RFC 1035): a UDP answer is at most **512 bytes**.
2. **EDNS** (RFC 6891): the client announces a bigger buffer (`udp: 1232` in dig).
3. **DNS Flag Day 2020**: 1232 bytes, to avoid IP fragmentation.
4. Too big anyway? The server cuts the answer and sets `tc` (truncated). The client asks again **over TCP**, which every server must support (RFC 7766).

```
❯ dig @8.8.8.8 google.com TXT +noall +stats
;; MSG SIZE  rcvd: 1187
❯ dig @8.8.8.8 google.com TXT +noedns
;; Truncated, retrying in TCP mode.
;; SERVER: 8.8.8.8#53(8.8.8.8) (TCP)
```

PREDICT **Add `+ignore`: dig keeps the cut answer instead of retrying. What do you expect?**

```
❯ dig @8.8.8.8 google.com TXT +noedns +ignore
;; flags: qr tc rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
```

TRY IT `dig . DNSKEY +bufsize=512 +ignore`: is the root's key set too big for 512 bytes?

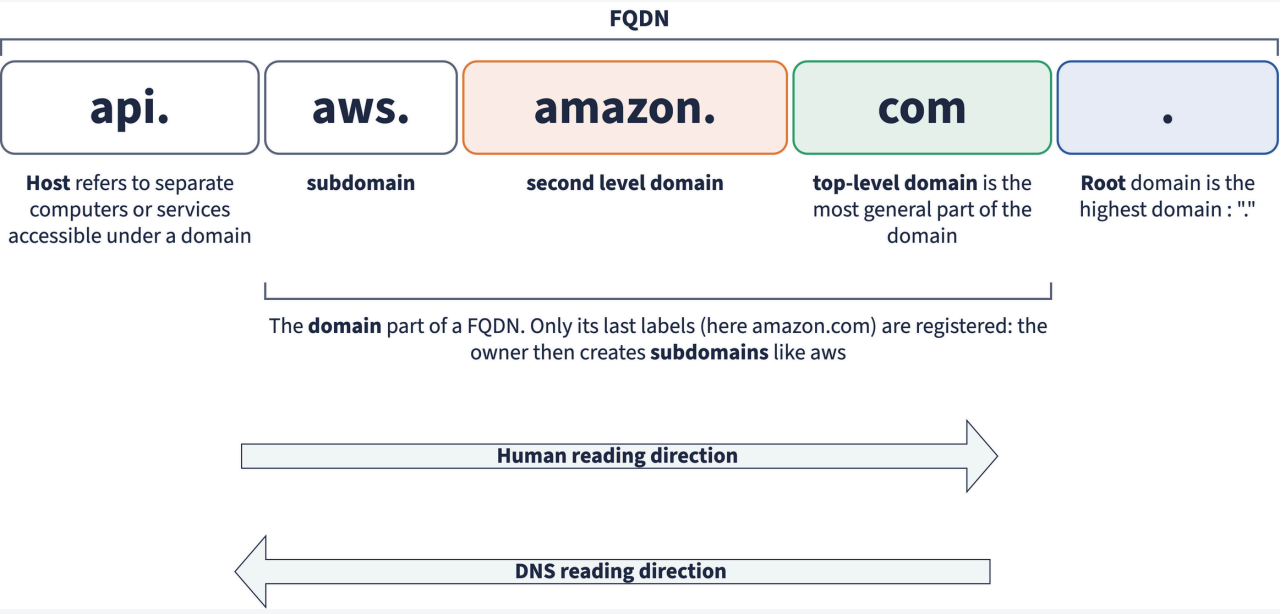
Long TXT records

A TXT record is a list of **strings of at most 255 bytes each**. Longer data (a 2048-bit DKIM key, a big SPF policy) is split. Programs that read it **join the strings with nothing in between**; dig shows each string in its own quotes.

```
❯ dig TXT whoami.ds.akahelp.net +short
"ns" "141.101.96.152"            # ONE record, TWO strings
"ecs" "86.201.70.0/24/24"
❯ dig TXT dns.google +short
"v=spf1 -all"                    # two records, one string each
"https://xkcd.com/1361/"
```

- Several strings in **one record**: one value, cut in pieces → join them.
- Several **records**: separate values, in no fixed order.

TRY IT `dig TXT google.com +short`: find the longest value. Is it one string or several?



PART 4

Special records and names

Null MX, mail authentication, apex, SRV, NAPTR, IPv6 reverse, IDN

Null MX: “no mail here”

A domain **without** MX still receives mail: senders fall back to its A/AAAA address (RFC 5321) and keep retrying for days.

A **null MX** (RFC 7505) says it clearly: this domain accepts **no mail**. Senders fail at once and the user gets a clear bounce.

It goes with an SPF `v=spf1 -all`: “this domain never **sends** mail either”.

```
❯ dig example.com MX +short
0 .
❯ dig iana.org MX +short
10 pechora1.icann.org.
10 pechora6.icann.org.
…
```

`0 .`: priority 0, target `.` (the root, which is not a host).

TRY IT `dig example.org MX +short` and `dig example.org TXT +short`: does example.org send mail?

Mail authentication: SPF, DKIM, DMARC

SPF

TXT on the domain: `v=spf1 include:… -all`. **Which servers** may send mail for this domain.

DKIM

TXT at `<selector>._domainkey.<domain>`: the **public key** that checks the mail's signature. The selector is in the mail header (`DKIM-Signature: … s=selector`).

DMARC

TXT at `_dmarc.<domain>`: **what to do** when SPF and DKIM fail (`p=none`, `quarantine`, `reject`) and where to send reports (`rua=`).

```
❯ dig _dmarc.airbus.com TXT +short
reject.dmarc.airbus.com.                    # a CNAME: one shared policy
"v=DMARC1;p=reject;rua=mailto:…;ruf=mailto:…;fo=1;"
```

TRY IT `dig _dmarc.gouv.fr TXT +short`: what does `sp=none` change for the subdomains?

No CNAME at the zone apex

A CNAME says “this name is **only** an alias”, so no other record may sit next to it.

The apex (`example.com` itself) **always** has SOA and NS records → **it can’t be a CNAME**.

So how do `example.com` sites point to a CDN?

ALIAS / ANAME / flattening

The DNS provider looks up the target itself and serves plain A/AAAA records. Not a standard: each provider has its own name.

HTTPS / SVCB

Standard record (RFC 9460): it can point to another name, gives ports, HTTP/3, IP hints.

```
❯ dig cloudflare.com HTTPS +short
1 . alpn="h3,h2" ipv4hint=104.16.132.229,104.16.133.229 ipv6hint=2606:4700::6810:84e5,…
```

TRY IT `dig www.airbus.com CNAME +short`, then `dig airbus.com CNAME +short`. Why the difference?

SRV and NAPTR: services, ports, rewriting

SRV (RFC 2782) finds a service: `_service._protocol.domain` → **priority, weight, port, host**.

```
❯ dig _imaps._tcp.gmail.com SRV +short
5 0 993 imap.gmail.com.        # priority 5, weight 0, port 993
❯ dig _xmpp-server._tcp.jabber.org SRV +short
30 30 5269 scarlet.jabber.org.
```

NAPTR (RFC 3403) rewrites a name with a **regular expression**. ENUM uses it to turn a phone number into a SIP address: digits reversed under `e164.arpa`.

```
❯ dig 4.4.2.2.3.3.5.6.8.1.4.4.e164.arpa NAPTR +short     # +44 1865 332244
100 10 "u" "E2U+sip" "!^\\+441865332(.*)$!sip:\\1@nominet.org.uk!" .
```

TRY IT Apply the regex to `+441865332244`: which SIP address do you get? (`\1` is what `(.*)` matched.)

Reverse lookup in IPv6

IPv4 reverses the 4 bytes under `in-addr.arpa`. IPv6 reverses **every hex digit** (a “nibble”) under `ip6.arpa` (RFC 3596): 32 labels.

Step	2001:4860:4860::8888
1. Write all 32 digits (expand `::`)	`2001 4860 4860 0000 0000 0000 0000 8888`
2. Reverse them, one per label	`8.8.8.8.0.0.0.0.0.0.0.0.0.0.0.0.0.6.8.4.0.6.8.4.1.0.0.2`
3. Append the zone	`….1.0.0.2.ip6.arpa.`

```
❯ dig -x 2001:4860:4860::8888 +short
dns.google.
```

TRY IT Build the name by hand, then check: `dig PTR <your name> +short` must give the same answer.

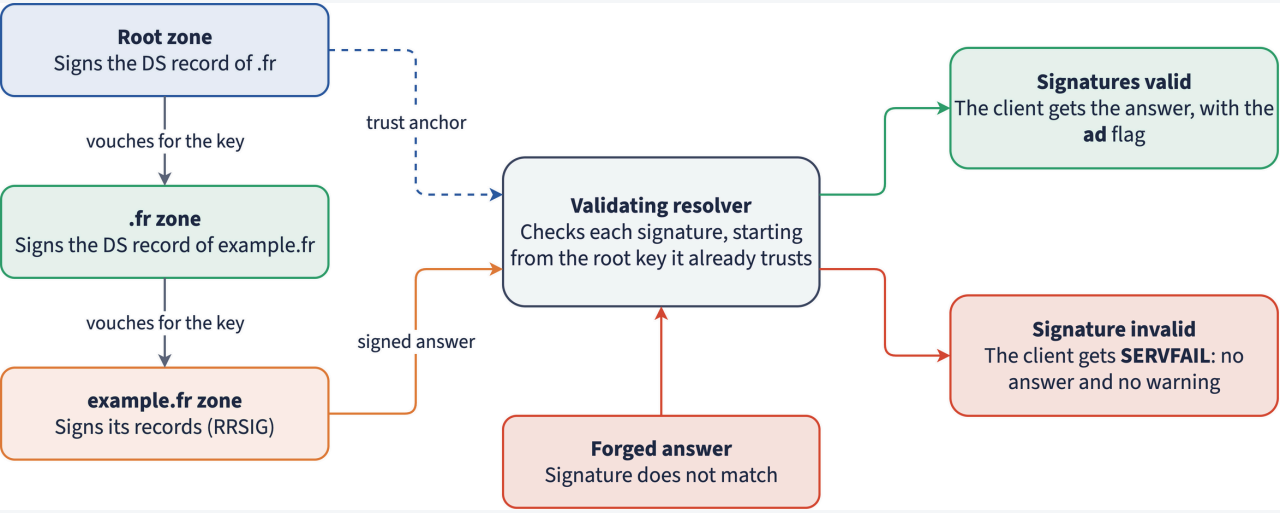
Names beyond ASCII: IDN

DNS labels stay plain ASCII (letters, digits, hyphen). A Unicode name is converted to **Punycode** (RFC 3492) with the prefix `xn--` (IDNA, RFC 5890–5893). Browsers show the Unicode form, DNS only sees the `xn--` form.

You type	DNS sees
résumé.fr	`xn-rsum-bpad.fr`
café.fr	`xn-caf-dma.fr`
bücher.de	`xn-bcher-kva.de`
straße.de	`xn-strae-oqa.de` (IDNA 2008) or `strasse.de` (2003)

The `ß` case shows two standards giving **two different domains** for the same text. Lookalike letters (Cyrillic `a` vs Latin `a`) give the homograph attacks seen in *DNS Fundamentals*.

TRY IT `dig xn--caf-dma.fr +short`, then `dig café.fr +short` (recent dig converts it for you).



PART 5

DNSSEC in practice

The records, the chain of trust, the flags

The four records of DNSSEC

`DNSKEY`	The zone's public keys. Flags `257` = **KSK** (key-signing key, signs the keys), `256` = **ZSK** (zone-signing key, signs the records).
`RRSIG`	The signature of one record set: algorithm, labels, validity dates, **key tag** of the key that signed it.
`DS`	Stored in the **parent** zone: a hash of the child's KSK. It links the child to the parent's signature.
`NSEC` `NSEC3`	A signed proof that a name or a type **does not exist** (part 6).

TRY IT `dig fr DNSKEY +dnssec +multi`: find the KSK, the ZSK and the RRSIG that signs them.

Following the chain: DS → DNSKEY

```
❯ dig fr DS +short                                  # asked to the root's data
65381 13 2 8163ABF45792942CF4EE38CCA31F6A6832FCDC6D402338FC687827690C4132F6
❯ dig fr DNSKEY +multi +noall +answer               # asked to .fr's data
fr.  163 IN DNSKEY 256 3 13 ( Z4ReGON4… ) ; ZSK; alg = ECDSAP256SHA256 ; key id = 1580
fr.  163 IN DNSKEY 257 3 13 ( C8Q2nWGQ… ) ; KSK; alg = ECDSAP256SHA256 ; key id = 65381
```

- DS fields: **key tag** `65381`, **algorithm** `13` (ECDSA P-256), **digest type** `2` (SHA-256), then the hash.
- The key tag points to the **KSK** of `.fr`. The root signs the DS, `.fr`'s KSK signs its ZSK, the ZSK signs the records.
- Your resolver repeats this up to the **root key**, which it knows in advance (the *trust anchor*).

TRY IT Do the same one level down: `dig afnic.fr DS +short` (asked to `.fr`), then `dig afnic.fr DNSKEY`.

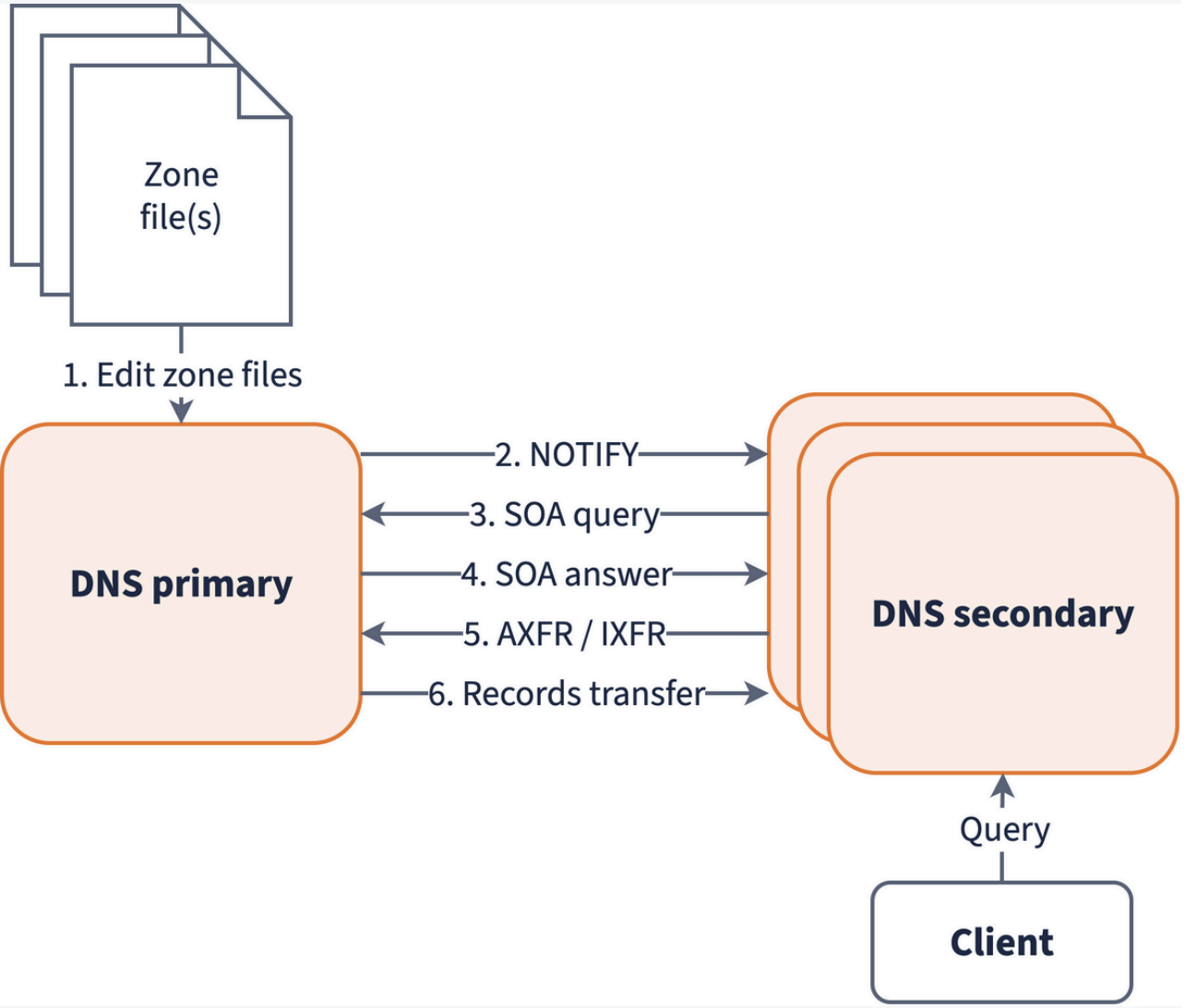
The flags: `do`, `ad`, `cd`

do	**Client:** send me the signatures (`+dnssec`)
ad	**Resolver:** I checked it, it is valid
cd	**Client:** don't check, give me the data (`+cd`)

```
❯ dig dnssec-failed.org @8.8.8.8
;; status: SERVFAIL       ;; flags: qr rd ra
; EDE: 9 (DNSKEY Missing): (No DNSKEY matches DS RRs of dnssec-failed.org)
❯ dig dnssec-failed.org @8.8.8.8 +cd +short
96.99.227.255
```

- A **bogus** signature gives `SERVFAIL`, the same status as a dead server. `+cd` tells them apart: if the data comes back, DNSSEC was the problem.
- `ad` is set **by the resolver**: you trust it, and the path to it.

TRY IT `dig iana.org @1.1.1.1` vs `dig iut-blagnac.fr @1.1.1.1`: which one has the `ad` flag? Why?



PART 6

Opening a zone

Zone transfers and walking the NSEC chain

Zone transfers: AXFR

Secondary servers copy the zone from the primary:

- **AXFR**: the whole zone, over TCP, between two SOA records
- **IXFR**: only the changes since a serial (RFC 1995)
- **NOTIFY**: “the serial changed, come and get it”

If **anyone** may run an AXFR, the whole zone leaks: every host name, internal naming, test servers. Allow it only to the secondaries (address list), and sign transfers with **TSIG** (a shared key, RFC 8945).

```
❯ dig @nsztm1.digi.ninja zonetransfer.me AXFR
zonetransfer.me. 7200 IN SOA nsztm1.digi.ninja. …
zonetransfer.me. 301  IN TXT "google-site-…"
zonetransfer.me. 7200 IN MX  0 ASPMX.L.GOOGLE.COM.
zonetransfer.me. 7200 IN A   5.196.105.14
zonetransfer.me. 7200 IN NS  nsztm1.digi.ninja.
…
```

`zonetransfer.me` is left open **on purpose** for training. Try AXFR only on zones you are allowed to test.

TRY IT Run the AXFR above. How many records? Which ones would interest an attacker?

Proving that a name does not exist: NSEC

A server can't sign "nothing". So it signs the **gap**: an NSEC record says "after this name, the next existing name is X, and nothing is in between". Ask for each next name, and you **walk the whole zone**, without AXFR.

```
❯ dig isc.org NSEC +short
_acme-challenge.isc.org. A NS SOA MX TXT AAAA NAPTR RRSIG NSEC DNSKEY CDS CDNSKEY HTTPS CAA …
❯ dig _acme-challenge.isc.org NSEC +short
_atproto.isc.org. CNAME RRSIG NSEC
❯ dig _atproto.isc.org NSEC +short
_dmarc.isc.org. TXT RRSIG NSEC
❯ dig _dmarc.isc.org NSEC +short
05DFB016-….._domainkey.isc.org. TXT RRSIG NSEC      # a DKIM selector, found without guessing
```

TRY IT Walk three more steps. Then: why would a company **not** want this?

NSEC3 and compact denial

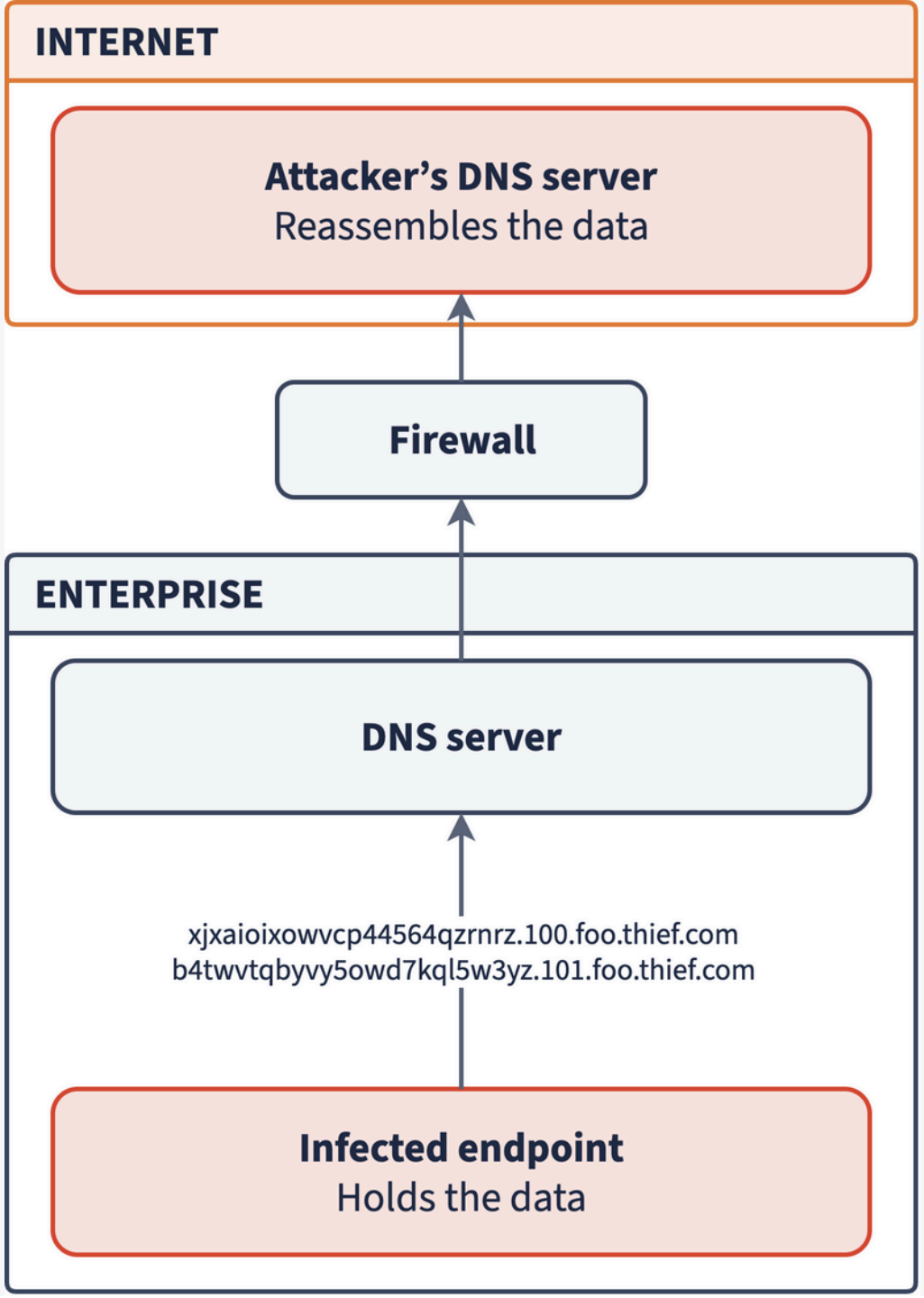
NSEC3 (RFC 5155) chains **hashes** of names instead of names. Walking gives hashes only… but common names can be guessed offline with a dictionary.

Compact denial (Cloudflare, Route 53; RFC 9824) signs a tiny gap built on the fly, from the name to `\000.name`. Nothing is revealed, and the answer becomes `NOERROR` with a type `NXNAME` (`TYPE128`) instead of `NXDOMAIN`.

```
❯ dig zz-nope.iana.org +dnssec +noall +authority
2su40cd1….iana.org. 3600 IN NSEC3 1 0 0 -
  4DI2M49H… AAAA RRSIG
#  hash 1 = SHA-1, 0 iterations, no salt

❯ dig zz-nope.cloudflare.com +dnssec
;; status: NOERROR
zz-nope.cloudflare.com. 300 IN NSEC
  \000.zz-nope.cloudflare.com.
  RRSIG NSEC TYPE128
```

TRY IT `dig zz-nope.afnic.fr +dnssec +noall +authority`: NSEC or NSEC3? With a salt?



PART 7

DNS as a puzzle

Encodings, chained clues and files in pieces

Data hidden in TXT records

TXT holds any bytes, so CTF challenges (and malware, see *infiltration* in *DNS Fundamentals*) hide data in it. Recognise the encoding, then decode it:

Looks like	Encoding	Decode with
`"ZGlnIGRlZXBlcg=="`	Base64: A–Z a–z 0–9 + /, ends with `=`	`echo ZGlnIGRlZXBlcg== | base64 -d`
`"646967"`	Hex: 0–9 a–f, even length	`echo 646967 | xxd -r -p`
`"\240\159\148\141"`	dig's escape for bytes it can't print: **decimal**	`printf '\360\237\224\215'` (printf wants octal)
`"\144\151\147"` inside the data	Octal written by the author: **check the digits** (no 8 or 9)	`printf '\144\151\147'`

TRY IT `dig TXT wd.ip.wtf +short`: Wordle over DNS. Its second record shows a shell function that decodes dig's escapes.

Chained clues and files in pieces

CHAIN

Each record names the next one to ask: a TXT points to an MX, the MX host has an SRV, the SRV gives a port…

MAZE

Several CNAMEs at each step, only one leads somewhere. A loop in a script beats clicking.

CHUNKS

A file split into `chunk-0` … `chunk-N` TXT records, plus a record with the count and a SHA-256 to check the result. Real malware stages payloads this way.

```
❯ for i in $(seq 0 9); do dig +short TXT chunk-$i.<zone>; done | tr -d '" \n' | base64 -d > out.bin
❯ shasum -a 256 out.bin                          # compare with the meta record
```

TRY IT The Advanced missions on the game zone use these three patterns.

LAB

Dig Quest: Advanced missions



Dig Quest Advanced missions

Cheat sheet

- They unlock once **80% of the main missions** are solved.
- A **separate leaderboard**: the main ranking stays as it was.
- The dig console gains `class CH`, `+nsid`, `+cd`, and AXFR and `-p` toward the game server.
- Hints cost points as before. Each mission names the part of this deck it needs.
- The explanations unlock when you solve them all, or when the session closes.

What to remember

1. Anycast hides **which** server answered: `CH TXT`, `+nsid` and whoami names reveal it.
2. A resolver is a chain: the authoritative server sees the last hop, and maybe your subnet (ECS).
3. Read the SOA and the EDE text before guessing why a lookup failed.
4. Wildcards answer for names that don't exist; `+norecurse` reads a resolver's cache.
5. Big answers set `tc` and move to TCP; long TXT values come in 255-byte pieces.
6. Null MX, SPF, DKIM and DMARC are all plain DNS records: you can audit a domain's mail with dig.
7. DNSSEC is a chain DS → DNSKEY → RRSIG up to the root. `SERVFAIL` + `+cd` = a DNSSEC problem.
8. AXFR and NSEC walking both list a zone: restrict the first, prefer NSEC3 or compact denial for the second.

Resources

RFC 4892 (`id.server`), RFC 5001 (NSID), RFC 7108 (L-root identity), RFC 8914 (Extended DNS Errors)

RFC 4592 (wildcards), RFC 6891 (EDNS), RFC 7766 (DNS over TCP), RFC 7505 (null MX)

RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC), RFC 2782 (SRV), RFC 3403 (NAPTR), RFC 9460 (SVCB/HTTPS)

RFC 3596 (IPv6 in DNS), RFC 3492 (Punycode), RFC 4033–4035 (DNSSEC), RFC 5155 (NSEC3), RFC 9824 (compact denial)

Mess With DNS (Julia Evans) · zonetransfer.me · Why you can't dig Switzerland

Which anycast server are you talking to? (jpmens) · RFC search